

State Chart Diagrams

Software Engineering Lab — Student Handout

1. What Is a State Chart Diagram?

A **state chart diagram** (also called a **state machine diagram**) is a UML behavioral diagram that shows the life of a single object: all the **states** (situations or conditions) the object can be in, and the **events** that move it from one state to another. An object in a given state waits until something happens; when an event occurs, the object may **transition** to a new state and perform some action along the way. For example, an ATM can be *Off*, *Idle*, *Serving a Customer*, or *Out of Service*, and events such as *cardInserted* or *failure* move it between these states. State charts are most useful when an object reacts differently to the same event depending on its current state, for instance, an ATM ignores a card when it is switched off but accepts it when it is idle. By making every state and every event explicit, the diagram helps developers find missing cases before coding, gives testers a clear basis for test cases (one per transition), and gives everyone a shared picture of how the object should behave.

2. Basic Notation

The following symbols appear in almost every state chart diagram:

Element	How it is drawn	Meaning
Initial state	Small solid black circle	Where the object starts. Each diagram (and each region) has exactly one.
State	Rounded rectangle with a name	A situation in which the object waits or performs an activity.
Transition	Arrow from one state to another	A change of state, labelled trigger [guard] / effect.
Choice	Small diamond	A decision point; the outgoing transition whose guard is true is taken.
Final state	Solid circle inside a ring (bull's-eye)	The object (or region) has finished its behaviour.
Composite state	Large rounded rectangle containing other states	A state that has its own nested states inside it (see Section 5).

3. How Is It Different from Other Diagrams?

Each diagram answers a different question about the same system. A **use case diagram** shows *who* uses the system and *what* they can do with it (for example, a customer can withdraw cash), from the user's point of view, without showing anything inside the system. A **class diagram** shows the *structure* of the system: its classes, their attributes and methods, and how they are related (for example, *ATM*, *Card* and *Account*); it is static, like a snapshot, and says nothing about behaviour over time. A **data flow diagram** (DFD) shows *how data moves* between processes, data stores and external entities (for example, card details flowing to a verification process); it follows data rather than objects and does

not show order, timing or conditions, and unlike the other three it is not part of UML. A **state chart diagram**, in contrast, is dynamic and focuses on *one object* at a time, showing how that object's behaviour changes over its lifetime as events occur. These diagrams are not alternatives but complementary views: the use case diagram tells us what the system must do, the class diagram tells us which objects we need, the DFD shows how information flows, and the state chart picks one object from the class diagram and describes how it behaves.

4. Transitions: Trigger, Guard and Effect

A transition label has up to three parts, written in this form:

```
trigger [guard] / effect
```

All three parts are optional. A transition can have only a trigger, only a guard, both, or no label at all.

Trigger — what causes the transition

A **trigger** is the event that starts the transition. While in a state, the object waits; when a matching event arrives, the transition becomes a candidate to fire. Examples: `cardInserted`, `cancel`, `failure`. A special kind is the **time event**, such as `after(30s)`. A transition with *no* trigger is a **completion transition**: it fires automatically as soon as the source state has finished its work.

Guard — whether the transition is allowed

A **guard** is a Boolean condition written in square brackets. It is checked when the trigger occurs, and the transition fires only if the guard is true; otherwise the object stays in its current state. Example: after a self test, `[passed]` leads to *Idle* and `[failed]` leads to *Out of Service*. Guards on transitions leaving the same state should never be true at the same time.

Effect — what the system does during the transition

An **effect** is an action performed while the transition happens, after leaving the source state and before entering the target state. It is written after a slash. Example: in `turnOn / startup`, the ATM runs *startup* on its way from *Off* to *Self Test*.

Label	Trigger	Guard	Effect
<code>turnOn / startup</code>	<code>turnOn</code>	—	<code>startup</code>
<code>[passed]</code>	— (completion)	<code>passed</code>	—
<code>cardInserted</code>	<code>cardInserted</code>	—	—
<code>withdraw [balance ≥ amount] / dispenseCash</code>	<code>withdraw</code>	<code>balance ≥ amount</code>	<code>dispenseCash</code>

Order of execution when a transition fires

- 1 The trigger event occurs.
- 2 The guard is evaluated. If it is false, nothing happens and the object stays in its state.
- 3 The **exit activity** of the source state runs (if it has one).
- 4 The **effect** of the transition runs.
- 5 The **entry activity** of the target state runs (if it has one).

5. Composite States and Regions

Composite state

A **composite state** is a state that contains other states nested inside it. It is used to group related states that belong to one larger activity. In the ATM example, *Service Customer* is a composite state containing *Customer Authentication*, *Selecting Transaction* and *Transaction*. Its main benefit is simplicity: a transition drawn from the *border* of the composite state applies to every state inside it. So a single cancel arrow from *Service Customer* means the customer can cancel at any point, instead of drawing a separate cancel arrow from each inner state. When the inner flow reaches its final state, the composite state is complete, and an unlabelled completion transition leading out of it fires.

Region

A **region** is the area inside a composite state that holds its nested states and transitions. Each region has its own initial state and may have its own final state. A composite state usually has one region. If it has **two or more regions**, drawn as compartments separated by a dashed line, it is called an **orthogonal state**, and the regions run **at the same time**. For example, while the ATM handles the customer's menu in one region, a second region could monitor the card slot.

6. Internal Activities: Entry, Do and Exit

Activities can be attached to a state itself, rather than to a transition. They are written inside the state, below its name, in the form entry / action, do / activity and exit / action. They belong to the state, not to its region.

Activity	When it runs	ATM example
Entry activity	Once, every time the object <i>enters</i> the state, no matter which transition brought it in.	entry / readCard on Service Customer: the card is read as soon as serving begins.
Do activity	Continuously <i>while</i> the object is in the state. It can be interrupted by an event; when it finishes, any completion transition may fire.	do / runSelfTest on Self Test: the test runs while the ATM is in this state.
Exit activity	Once, every time the object <i>leaves</i> the state, no matter which transition takes it out.	exit / ejectCard on Service Customer: the card is always returned, whether the customer finishes, cancels, times out, or the machine fails.

Entry/exit activity vs. effect: an effect belongs to *one* transition, while entry and exit activities belong to a *state* and run on every way in or out. Use entry/exit when the same

action must happen regardless of how the state is entered or left.

7. Worked Example: Bank ATM

Figure 1 on the next page brings all of these ideas together. It models the behaviour of one ATM from the moment it is switched on. The page is printed sideways so the labels are easy to read.

How to read it

- The ATM starts in **Off**. On turnOn / startup it runs its startup routine and enters **Self Test**, where do / runSelfTest runs.
- When the test finishes, the guards decide the next state: [passed] leads to **Idle**, [failed] leads to **Out of Service**.
- From **Idle**, cardInserted enters the composite state **Service Customer**, and entry / readCard runs immediately.
- Inside, the customer authenticates. A choice sends them to **Selecting Transaction** if [authenticated], or to the final state if [authFailed]. After a **Transaction**, they can do [another Transaction] or finish.
- The ATM leaves Service Customer on completion, cancel, after(30s) or failure. In every case exit / ejectCard runs first.
- **Maintenance** is entered on service and left on serviceComplete, after which the ATM re-tests itself before serving customers again.
- *Customer Authentication* and *Transaction* are marked «submachine»: each is described in its own separate state chart to keep this diagram readable.

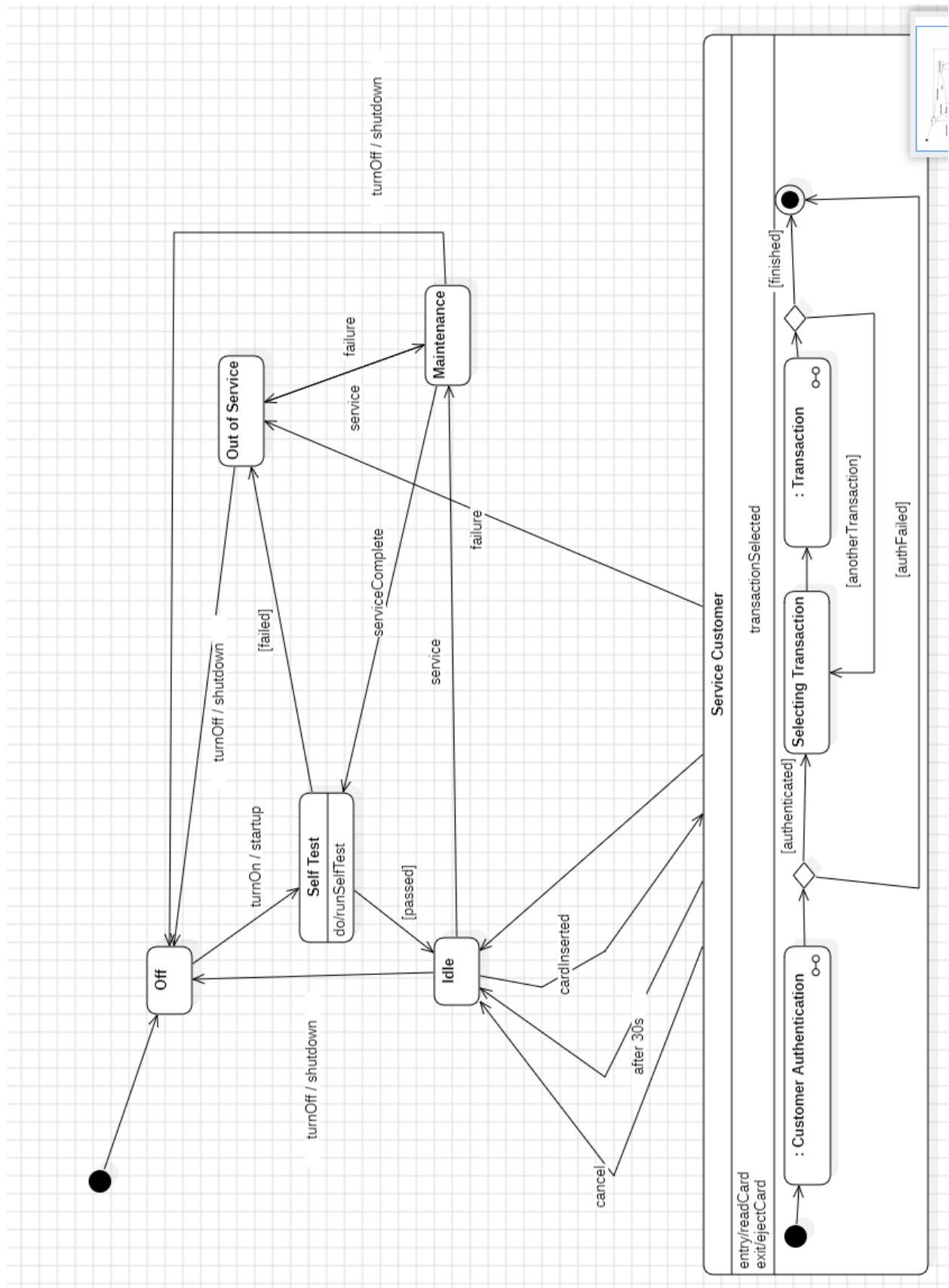


Figure 1. State chart diagram of a bank ATM (rotate the page to read).

8. Tips and Common Mistakes

- **Name states as conditions, not actions.** Use *Idle* or *Waiting for PIN*, not *Check PIN*. Actions belong on transitions or in activities.
- **Beware of empty completion transitions.** An unlabelled arrow out of a simple state with no do activity fires immediately, so the object never really stays in that state. Give the state a do activity or give the arrow a trigger.
- **Make guards non-overlapping.** If two outgoing guards can both be true, the diagram is ambiguous.
- **Check every state is reachable and can be left.** Ask “how do we get here?” and “how do we get out?” for each state.
- **Think about unusual events.** What happens on a timeout, a cancel or a failure? Missing arrows usually mean missing requirements.
- **Use composite states to avoid repeated arrows.** If many states share the same outgoing transition, group them.

9. Step-by-Step: Drawing the ATM Diagram in StarUML

Follow these steps in order to reproduce Figure 1. Menu names may differ slightly between StarUML versions; if an option is not where described, right-click the element and look in the **Add** submenu.

Part A — Set up the project

- 1 Open StarUML and create a new project with **File → New**.
- 2 In the **Model Explorer** (top right), right-click **Model** and choose **Add Diagram → Statechart Diagram**.
- 3 With the new diagram selected, rename it to Bank ATM in the **Properties** panel (bottom right). If the Properties panel is hidden, turn it on from the **View** menu.
- 4 Create the two submachines that will be referenced later. Right-click **Model** again, choose **Add Diagram → Statechart Diagram**, and name it Customer Authentication. Repeat to create Transaction. They can stay empty for this exercise.
- 5 Double-click Bank ATM in the Model Explorer to open it again before continuing.

Part B — Top-level states

- 1 Select **Initial State** in the Toolbox (left) and click near the top-left corner of the canvas.
- 2 Select **Simple State** in the Toolbox and drag a small box on the canvas. Type Off and press Enter.
- 3 Repeat to create four more simple states: Self Test, Idle, Maintenance and Out of Service. Arrange them roughly as in Figure 1, leaving the bottom of the canvas empty for the composite state.
- 4 Add the do activity to Self Test: right-click Self Test, choose **Add → Do Activity**, select **OpaqueBehavior**, type runSelfTest and press Enter. The state now shows do/runSelfTest in a compartment below its name.

Part C — The composite state

- 1 Select **Composite State** in the Toolbox and drag a wide box across the bottom of the canvas. Name it Service Customer.

- 2 Check that it has a region: in the Model Explorer, expand Service Customer and look for a child such as Region1. If there is none, right-click the state and choose **Add → Region**. The area below the horizontal divider line is the region.
- 3 Right-click on the **name** of Service Customer (not inside the region) and choose **Add → Entry Activity → OpaqueBehavior**. Name it readCard.
- 4 Right-click the name again and choose **Add → Exit Activity → OpaqueBehavior**. Name it ejectCard. Both activities appear in the compartment between the state name and the region, and belong to the state, not to the region.

Part D — States inside the composite state

Create each of the following **directly inside the region**, below the divider line. StarUML only nests elements that are *created* inside the region. Dragging an existing element into the box moves the shape but does not nest it in the model.

- 1 Select **Initial State** in the Toolbox and click at the left edge of the region.
- 2 Select **Submachine State** in the Toolbox and drag a box to the right of the initial state. In the Element Picker dialog, choose the Customer Authentication state machine. Leave the state's own name blank; StarUML displays it as : Customer Authentication with the submachine icon.
- 3 Select **Choice** and click to the right of Customer Authentication.
- 4 Select **Simple State** and drag a box. Name it Selecting Transaction.
- 5 Select **Submachine State**, drag a box, and choose the Transaction state machine. It displays as : Transaction.
- 6 Select **Choice** again and click to the right of Transaction.
- 7 Select **Final State** and click at the right edge of the region.
- 8 Verify the nesting in the Model Explorer. Under Service Customer → Region1 you should see the initial state, both submachine states, both choices, Selecting Transaction and the final state. If any element is listed elsewhere, drag it in the Model Explorer onto Region1, or delete it with **Edit → Delete from Model** and recreate it inside the region.

Part E — How to draw and label a transition

Use this procedure for every transition in Parts F to H:

- **Draw:** select **Transition** in the Toolbox, press on the source element and drag to the target element.
- **Trigger:** double-click the transition to open Quick Edit, click **Add Trigger Event**, and type the event name. Then select the new event in the Model Explorer (it is listed under the transition) and set **kind** in the Properties panel: signal for ordinary events, time for after(30s). Do not leave it as anyReceive.
- **Guard:** select the transition and type the condition in the **guard** field of the Properties panel, *without* square brackets. StarUML adds the brackets on the diagram.
- **Effect:** double-click the transition, click **Add Effect Behavior**, choose **OpaqueBehavior** and type the name.
- **Tidy:** drag the label away from the line if it overlaps anything. To make right-angled lines, select the transition and choose **Format → Line Style → Rectilinear**.

Part F — Top-level transitions

From	To	Trigger (kind)	Guard	Effect
Initial State	Off	—	—	—
Off	Self Test	turnOn (signal)	—	startup
Self Test	Idle	—	passed	—
Self Test	Out of Service	—	failed	—
Idle	Off	turnOff (signal)	—	shutdown
Idle	Maintenance	service (signal)	—	—
Maintenance	Self Test	serviceComplete (signal)	—	—
Maintenance	Out of Service	failure (signal)	—	—
Maintenance	Off	turnOff (signal)	—	shutdown
Out of Service	Maintenance	service (signal)	—	—
Out of Service	Off	turnOff (signal)	—	shutdown

Part G — Transitions to and from the composite state

Connect these to the **border** of the Service Customer box, not to a state inside it.

From	To	Trigger (kind)	Guard	Effect
Idle	Service Customer	cardInserted (signal)	—	—
Service Customer	Idle	cancel (signal)	—	—
Service Customer	Idle	after(30s) (time)	—	—
Service Customer	Out of Service	failure (signal)	—	—
Service Customer	Idle	— (leave blank)	—	—

The last transition has **no label at all**. It is a completion transition: it fires automatically when the flow inside Service Customer reaches its final state. Do not add a trigger, guard or effect to it.

Part H — Transitions inside the composite state

None of these transitions has an effect.

From	To	Trigger (kind)	Guard
Initial State	Customer Authentication	—	—
Customer Authentication	Choice 1	—	—
Choice 1	Selecting Transaction	—	authenticated
Choice 1	Final State	—	authFailed
Selecting Transaction	Transaction	transactionSelected (signal)	—
Transaction	Choice 2	—	—
Choice 2	Selecting Transaction	—	anotherTransaction
Choice 2	Final State	—	finished

Part I — Check and export

- 1 Compare every label with Figure 1 and with the tables above. Check that the completion transition has no label and that each trigger has the correct kind.
- 2 In the Model Explorer, confirm that readCard and ejectCard are listed directly under Service Customer, and that the inner states are under its region.
- 3 Save the project with **File → Save** (StarUML saves as a .mdj file).
- 4 Export the picture with **File → Export Diagram As → PNG** (or SVG). Exporting gives a clean image without the grid or the navigator thumbnail that appear in a screenshot.